



BRENT OZAR
UNLIMITED®

When PAGEIOLATCH Is Your Top Wait

Deck 3 p1

Let's take this AWS RDS instance:

```
1 sp_BlitzFirst @SinceStartup = 1
```

150 % No issues found

Results Messages

	Pattern	Hours Sample	Thread Time (Hours)	wait_type	wait_category	Wait Time (Hours)	Avg ms Per Wait
1	WAIT STATS	218.0	2493.7	PAGEIOLATCH_SH	Buffer IO	1485.1	52.2
2	WAIT STATS	218.0	2493.7	CXCONSUMER	Parallelism	748.2	305.9
3	WAIT STATS	218.0	2493.7	CXSYNC_PORT	Other	214.7	52656.7
4	WAIT STATS	218.0	2493.7	ASYNC_NETWORK_IO	Network IO	15.4	137.5
5	WAIT STATS	218.0	2493.7	LCK_M_S	Lock	9.6	264472.7
6	WAIT STATS	218.0	2493.7	SOS_SCHEDULER_YIELD	CPU	6.3	1.7
7	WAIT STATS	218.0	2493.7	CXPACKET	Parallelism	3.5	4.7

Deck 3 p2



Top wait by far: PAGEIOLATCH

Hours Sample	Thread Time (Hours)	wait_type	wait_category	Wait Time (Hours)	Avg ms Per Wait
218.0	2493.7	PAGEIOLATCH_SH	Buffer IO	1485.1	52.2
218.0	2493.7	CXCONSUMER	Parallelism	748.2	205.9

Means SQL Server is waiting to read uncached data pages up from storage.

Avg ms Per Wait = when we wait for a read, this is how long we're waiting, on average. (And storage may be much slower than this!)

Deck 3 p3



Preparation for the demo

Set your max memory to 1GB, parallelism to defaults:

```
EXEC sys.sp_configure N'max server memory (MB)', N'1024';  
EXEC sys.sp_configure N'cost threshold for parallelism', N'5';  
EXEC sys.sp_configure N'max degree of parallelism', N'0';
```

Restart your SQL Server

Drop all nonclustered indexes on
StackOverflow.dbo.Comments

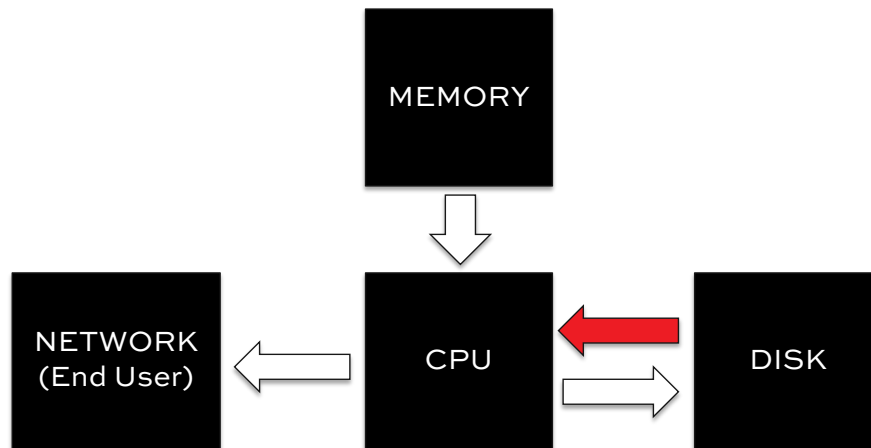
Set up a new window for sp_BlitzFirst:

```
sp_BlitzFirst @ExpertMode = 1, @Seconds = 30
```

Deck 3 p4



PAGEIOLATCH: Reading the data file from disk



Deck 3 p7



Data you have to read
/ How fast you read
Minimum query runtime

Deck 3 p8



Reading millions of 8KB pages

```
SELECT * FROM dbo.Comments WHERE UserId = 26837;
```

150 %

Results Messages

(150 rows affected)

Table 'Comments'. Scan count 9, logical reads 2765260,
physical reads 13, read-ahead reads 2762692,
lob logical reads 0, lob physical reads 0, lob read-ahead reads 0.

$2,765,260 \text{ pages} * 8\text{KB each} / 1,024 = 21.6\text{GB}$

Deck 3 p9



```
sp_BlitzFirst @ExpertMode = 1, @Seconds = 60
```

150 %

Results Messages

run_date	elapsed_time	session_id	database_name	query_text	query_plan	live_query_plan	query_cost	status	wait_info	top_session_waits	blocking_session_id	open_trar
----------	--------------	------------	---------------	------------	------------	-----------------	------------	--------	-----------	-------------------	---------------------	-----------

	Priority	FindingsGroup	Finding	Details	HowToStopIt	CheckID
1	0	sp_BlitzFirst 2018-...	From Your Community Volunteers	Click To See Details -- We hope you found this tool ...	NULL	-1
2	200	Wait Stats	CLR_AUTO_EVENT	Click To See Details -- For 117 seconds over the last ...	Click To See Command -- WARNING: Running t...	6
3	200	Wait Stats	PAGEIOLATCH_SH	Click To See Details -- For 55 seconds over the last ...	Click To See Command -- WARNING: Running t...	6
4	250	Server Info	Batch Requests per Sec	Click To See Details -- 0 -- ?>	NULL	19
5	250	Server Info	CPU Utilization	Click To See Details -- 11% Ring buffer details: <Re...	NULL	23
6	250	Server Info	SQL Compilations per Sec	Click To See Details -- 0 -- ?>	NULL	25
7	250	Server Info	SQL Re-Compilations per Sec	Click To See Details -- 0 -- ?>	NULL	26
8	250	Server Info	Wait Time per Core per Sec	Click To See Details -- 0 -- ?>	NULL	19

Pattern	Sample Ended	Seconds Sample	wait_type	wait_category	Wait Time (Seconds)	Avg ms Per Wait	Per Core Per Secor
1	WAIT STATS 2018-07-22 14:25:04.5936065 +00:00	59	CLR_AUTO_EVENT	SQL CLR	117.5	8394.3	0.2
2	WAIT STATS 2018-07-22 14:25:04.5936065 +00:00	59	PAGEIOLATCH_SH	Parallelism	55.6	4.0	0.1
3	WAIT STATS 2018-07-22 14:25:04.5936065 +00:00	59	CXCONSUMER	Parallelism	15.1	25.0	0.0
4	WAIT STATS 2018-07-22 14:25:04.5936065 +00:00	59	MEMORY_ALLOCATION_EXT	Memory	1.0	0.0	0.0
5	WAIT STATS 2018-07-22 14:25:04.5936065 +00:00	59	CXPACKET	Parallelism	0.6	14.5	0.0

Pattern	Sample Time	Sample (seconds)	File Name	Drive	# Reads/Writes	MB Read/Written	Avg Stall (ms)	file physical name
1	PHYSICAL READS 2018-07-22 14:25:04.6092026 +00:00	59	StackOverflow_4 [ROWS]	Z:	47209	5422.1	26	Z:\MSSQL\DAT
2	PHYSICAL READS 2018-07-22 14:25:04.6092026 +00:00	59	StackOverflow_3 [ROWS]	Z:	47392	5422.2	24	Z:\MSSQL\DAT
3	PHYSICAL READS 2018-07-22 14:25:04.6092026 +00:00	59	StackOverflow_2 [ROWS]	Z:	47359	5422.3	23	Z:\MSSQL\DAT
4	PHYSICAL READS 2018-07-22 14:25:04.6092026 +00:00	59	StackOverflow_1 [ROWS]	Z:	47319	5421.8	21	Z:\MSSQL\DAT
5	PHYSICAL WRITES 2018-07-22 14:25:04.6092026 +00:00	59	temp2 [ROWS]	Z:	8	0.1	6	Z:\MSSQL\DAT

So how do we tune this?

Data you have to read
/ How fast you read
Minimum query runtime

Deck 3 p11



How to reduce PAGEIOLATCH

1. Tune indexes, looking at missing indexes:
`sp_BlitzIndex @GetAllDatabases = 1`
2. Tune queries:
`sp_BlitzCache @SortOrder = 'reads'`
3. Add more memory
4. Make storage faster

Deck 3 p12



What index could you make?

```
/* Our query: */  
SELECT *  
FROM dbo.Comments  
WHERE UserId = 26837
```

Deck 3 p13



Fixing it with indexes

```
SELECT * FROM dbo.Comments  
WHERE UserId = 26837
```

```
/* Our index: */  
CREATE INDEX UserId ON  
dbo.Comments (UserId);
```

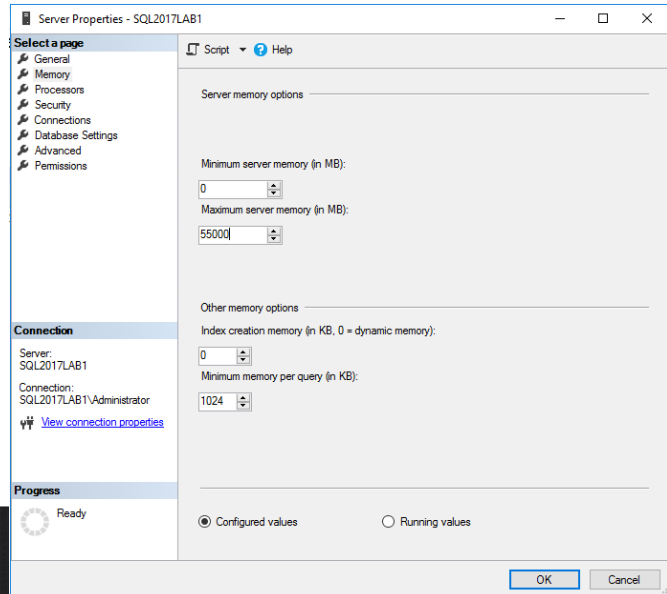
Deck 3 p14



Before you do that, add memory.

Otherwise, trust me,
the index build takes
forever.

My default: leave 4GB
or 10% free for the OS,
whichever is greater.



Deck 3 p15

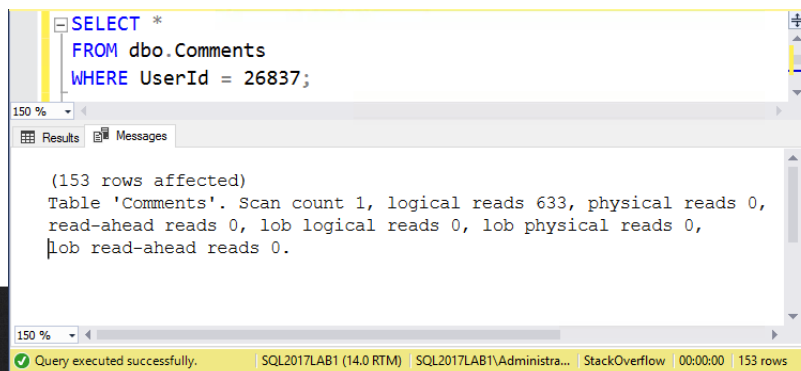
With the index, we read less pages

Under 1K pages down from millions, and finishes in a second

Frees up our memory for other queries

Reduces the workload on our slow storage

We don't have to cover the whole query – just one column helps



Deck 3

With the index, we read less pages

Pattern	Sample Ended	Seconds Sample	wait_type	wait_category	Wait Time (Seconds)	Per Core Per Second	Signal Wait Time (Seconds)	Percent Signal Waits	Number of Waits	Avg ms Per Wait
1 WAIT STATS	2018-02-13 22:33:30.8702564 +00:00	29	ASYNC_NETWORK_IO	Network IO	0.4	0.0	0.0	0.0	22	18.9
2 WAIT STATS	2018-02-13 22:33:30.8702564 +00:00	29	MEMORY_ALLOCATION_EXT	Memory	0.0	0.0	0.0	0.0	5391	0.0
3 WAIT STATS	2018-02-13 22:33:30.8702564 +00:00	29	PAGEIOLATCH_SH	Buffer IO	0.0	0.0	0.0	0.0	19	0.5
4 WAIT STATS	2018-02-13 22:33:30.8702564 +00:00	29	PREEMPTIVE_OS_WRITEFILE	Preemptive	0.0	0.0	0.0	0.0	1	1.0

Pattern	Sample Time	Sample (seconds)	File Name	Drive	# Reads/Writes	MB Read/Written	Avg Stall (ms)	file physical name
1 PHYSICAL READS	2018-02-13 22:33:30.8858867 +00:00	29	tempdev [ROWS]	Z:	3	0.2	0	Z:\MSSQL\DATA\tempdev.mdf
2 PHYSICAL READS	2018-02-13 22:33:30.8858867 +00:00	29	MSDBData [ROWS]	D:	8	0.5	0	D:\MSSQL14\MSSQLSERVER\MSSQL\DATA\MSDBData.mdf
3 PHYSICAL READS	2018-02-13 22:33:30.8858867 +00:00	29	StackOverflow_1 [ROWS]		38	2.4	0	Z:\MSSQL\DATA\StackOverflow_1.mdf

Waits down under 1 second.

The quality of the storage simply no longer matters.

Storage stalls disappear because less work means faster average response time per task.

Deck 3 p17



How to reduce PAGEIOLATCH

1. ~~Tune indexes, looking at missing indexes:~~

~~sp_BlitzIndex @GetAllDatabases = 1~~

2. Tune queries:

sp_BlitzCache @SortOrder = 'reads'

3. Add more memory

4. Make storage faster

But if we can't tune indexes

Deck 3 p18



I'd rather tune indexes first.

The right indexes make *lots* of queries faster.

Tuning individual queries can take a lot longer.

Deck 3 p19



Can we “fix” this query?

```
SELECT * FROM dbo.Comments  
WHERE UserId = 26837
```

1. Don't select columns you don't need
2. Don't select rows you don't need (paginate)
3. Cache in the application layer

We cover these in Mastering Query Tuning.

Deck 3 p20



How to reduce PAGEIOLATCH

1. ~~Tune indexes, looking at missing indexes:~~

~~sp_BlitzIndex @GetAllDatabases = 1~~

2. ~~Tune queries:~~

~~sp_BlitzCache @SortOrder = 'reads'~~

3. Add more memory

Now we're down to these.

4. Make storage faster

Deck 3 p21



But cloud hardware is different.

Let's take a break, and then dig into that.

Deck 3 p22

